Arithmetic, Answer Formatting, and Generalization in a Tiny Language Model Trained with GRPO

Mohamed Sobhy

Living experiment report — October 2026

Abstract

This project investigates direct reinforcement learning on a pretrained 0.8-billion-parameter language model for Countdown arithmetic. A pilot run improved strict greedy accuracy from 0/256 to 85/256 (33.2%) on a fixed held-out set after 300 optimizer steps. This report distinguishes measured findings from planned experiments. A post-hoc audit using a conservative declared-expression parser recovers the same correct counts when the answer-tag requirement is relaxed. This supports investigating arithmetic task improvement, while not establishing general reasoning or fully identifying the role of formatting. Increasing the pilot's output cap from 256 to 1024 tokens gained one solution while costing 3.85 times the evaluation time. Three completed three-number-only training seeds average 57.3% on three-number puzzles and 11.3% on four-number puzzles, with sample standard deviations of 5.5 and 3.1 percentage points. The live demo passed real CUDA and browser checks; the six-run reward comparison remains incomplete. Results below are explicitly scoped to their checkpoints and suites.

1 Question and experimental scope

When does direct GRPO improve arithmetic task performance, and how much of an apparent improvement is answer-format compliance? The base checkpoint is `Qwen/Qwen3.5-0.8B-Base`; the dataset is `Jiayi-Pan/Countdown-Tasks-3to4`. The experiment omits supervised fine-tuning. It does not train a model from scratch and does not claim discovery of general reasoning. Model and dataset revisions, code, and outputs are retained in the public repository.

2 Pilot protocol

The text backbone is trained with BF16 rank-16 LoRA on one RTX 4090 Laptop GPU with 16 GB VRAM. GRPO uses eight candidates per puzzle, 32 completions per optimizer step, a learning rate of 10^{-5}, no KL penalty, and a 256-token completion limit. The 300-step run used seed 42 and took 5,135.9 seconds of training time (85.6 minutes), excluding setup and evaluation.

Puzzles are deduplicated by $(\text{target}, \text{sort}(\text{numbers}))$. A seeded hash partition reserves 2% for evaluation before subset selection. The available training subset contains 20,000 puzzles; evaluation uses 256 fixed puzzles. Both base and adapter evaluations use greedy decoding with the same output limit. No evaluation-based checkpoint selection was performed.

The strict reward is binary: one final `<answer>` block must contain a valid arithmetic expression using each supplied number exactly once. The verifier supports binary $+, -, \times, /$, parentheses, and exact rational arithmetic. It rejects number reuse/omission and unsafe syntax. Negative and fractional intermediate values are allowed. No formatting bonus was used.

3 Pilot results

Metric on 256 held-out puzzles	Base	GRPO adapter
Strict correct	0 (0.0%)	85 (33.2%)
Strict valid expressions	11 (4.3%)	90 (35.2%)
Declared-expression diagnostic correct	0 (0.0%)	85 (33.2%)

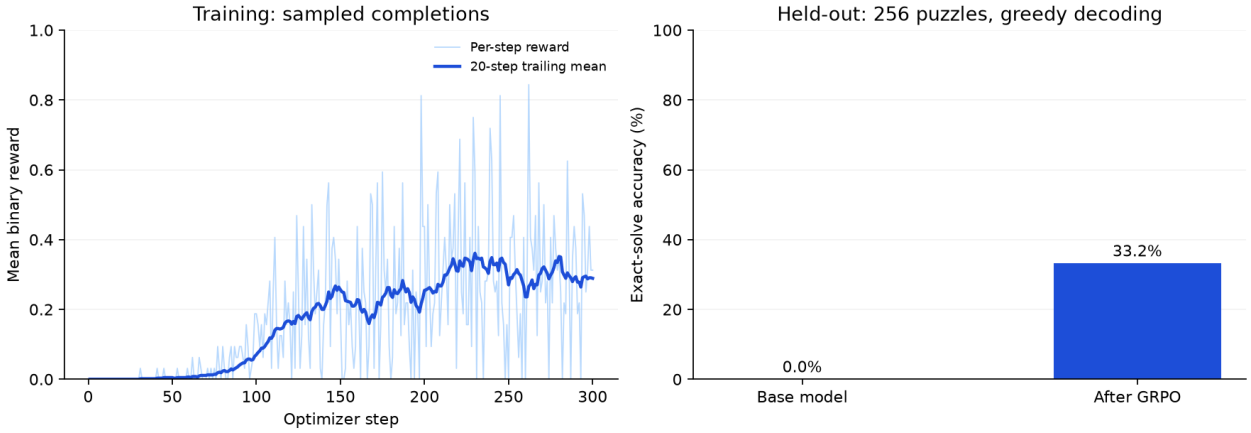


Figure 1: Pilot stochastic training reward and separate greedy held-out accuracy. These use different puzzles and decoding protocols; validation was not measured at every training step.

The first positive training reward occurred at step 31. Peak PyTorch-allocated memory was 2.72 GiB, which excludes reserved memory and other GPU allocations. In the final 30 training steps, 59.9% of sampled completions reached the output cap. Historical evaluation reports do not record termination token IDs, so their truncation status is unknown rather than inferred from decoded text.

4 Format-independent diagnostic: declared-expression-v1

This diagnostic is post-hoc and is not used as a training reward. Extraction is deterministic and independent of the target:

1. If answer tags occur, require exactly one complete block and extract it. A wrong tagged answer never falls back to an earlier expression.
2. Otherwise, select the last standalone arithmetic line, allowing an optional "Answer:" or "Final answer:" prefix and simple math/code delimiters.
3. For an equation, evaluate only the leftmost expression. Do not trust its claimed right-hand value or search intermediate calculations for a success.
4. Apply bounded AST interpretation, exact rational evaluation, and multiset validation of input numbers. Reject unsupported operations and syntax.

This conservative parser can miss correct answers expressed in prose or other notation. Therefore equal strict and diagnostic counts do not prove formatting had no causal role in training. They show that this specified relaxation did not recover additional correct answers from the archived generations.

Diagnostic outcome	Base	Adapter
Correct	0	85
Wrong numerical value with valid numbers	90	5
Number multiset mismatch	152	121
Invalid syntax	10	19
No declared expression	4	24
Ambiguous or incomplete answer tags	0	2

These are mutually exclusive diagnostic categories. Number omission/reuse details are retained per answer; truncation is a separate flag when token metadata exists. All 512 historical outputs were re-scored, and strict scores matched their original records. The same unique holdout puzzles were verified in both reports, with no intersection with the selected training subset. An independent exact subset-dynamic-programming solver found a valid witness for all 256 puzzles under the all-inputs-once rules. Unsolvable holdout items therefore do not explain the errors. Witnesses are archived separately and never included in model prompts. An agent review of the first two records per observed outcome category (20 records total) found agreement with the specified extraction rule; this is not a blinded human annotation study. Prose-heavy and incomplete outputs illustrate the parser's coverage limitations.

5 Controlled study design and current status

The study keeps the original strict metric alongside the diagnostic. The matched budget grid and mixed-arity pilot generalization grid are complete. All three transfer seeds and two of six controlled reward runs are complete:

- Matched base/adapter output budgets of 256, 512, and 1024 tokens; record actual generated lengths, EOS status, truncation, and generation time.
- Binary correctness versus a 0.05 bonus for valid incorrect expressions, with training seeds 42, 43, and 44 and a fixed split seed of 42.
- Three-number-only training with disjoint three- and four-number evaluation.
- Separately labeled, deterministic, solver-verified suites with larger inputs and out-of-range targets; solutions are never included in prompts.
- A comparison demo with exact verifier feedback and archived examples.

Per-seed results and held-out uncertainty will be reported separately. Four distribution-shift/arity suites have been frozen before querying trained models.

Uncertainty and paired analysis. Individual checkpoint accuracies use 95% Wilson score intervals, following the NIST formulation. For the pilot's 85/256 score this interval is 27.7–39.2%. It describes puzzle-sampling uncertainty under a binomial model, not training-seed variability or uncertainty about arbitrary out-of-distribution tasks. Comparisons match canonical puzzles and report gained, lost, jointly correct, and jointly incorrect answers. Exact two-sided McNemar tests are exploratory, with unadjusted p-values; we do not use the grid to claim a confirmatory significance result. The controlled-study aggregator requires all six runs to finish and checks their full training configuration, all 300 optimizer-step records, 20,000 unique training puzzles, identical ordered train/holdout lists, absence of overlap, adapter paths, and recomputed evaluation scores. It reports per-seed accuracies and sample standard deviations, together with paired reward differences for each seed. Repeated evaluation of the same holdout is not pooled into 768 independent puzzles.

Synthetic manifest tests reject unfinished runs and altered scores. The controlled aggregator additionally requires identical clean source, hardware, Python, normalized full package requirements, and pinned FLA versions across all six runs. The local transfer source/runtime audits are reported separately.

Interim controlled reward result. The first L4/FLA controlled run (binary reward, seed 42) completed 300 steps in 192.9 minutes and solves 68/256 held-out puzzles (26.6%; 95% Wilson interval 21.5–32.3%). Strict and diagnostic scores agree; 174/256 outputs are truncated. The downloaded archive's hash, full step sequence, settings, puzzle order, training/holdout separation, per-answer scores, and clean source/pinned-kernel provenance passed the single-run audit. The final adapter and resumable checkpoint are committed in W&B. Diagnostic outcomes comprise 68 correct expressions, 141 invalid syntax, 28 number mismatches, 10 without a declared expression, eight wrong values, and one ambiguous or incomplete answer. Truncation overlaps these categories. Evaluation evidence and raw outputs are archived under `results/study/controlled-progress`. The matched shaped seed-42 run has also completed; four runs remain pending. The historical laptop/fallback pilot used a different runtime environment, so its 85/256 score cannot isolate a hardware or kernel effect relative to this run.

The shaped seed-42 run completed 300 steps in 117.5 minutes and solves 26/256 puzzles (10.2%; Wilson 95% 7.0–14.5). All 256 declared expressions are valid under the all-inputs-once rules, but 230 have wrong values. None are truncated, and strict and diagnostic correct counts agree. Both runs passed full per-answer audits and have identical ordered puzzle lists, clean source, Python, full normalized requirements, optional kernels, and hardware metadata. Shaped reward gains 11 solutions and loses 53 relative to binary reward, with 15 jointly correct: a -16.4 percentage-point difference for this seed. This is improved expression validity alongside lower correctness, not a completed multi-seed conclusion. Raw training rewards differ in definition and cannot be compared directly. The remaining four controlled runs are pending. Shaped evaluation and the repository's `seed-42-comparison.json` preserve the evidence.

6 Completed output-budget experiment

The base model solved 0/256 at all three caps. The adapter solved 85/256 (33.2%) at 256 tokens and 86/256 (33.6%) at both 512 and 1024 tokens. Both longer caps gain the same single solution and lose none of the original successes. Strict and diagnostic scores coincide in every cell. The adapter's evaluation time increased from 184.6 to 358.6 to 710.0 seconds, while cap-reaching outputs declined only from 145 to 140 to 139 out of 256. Thus quadrupling the output cap costs approximately 3.85 times the evaluation time for one additional solution on this checkpoint and suite. These timings include batched generation and scoring, excluding model loading; they are not single-request latency. We do not infer that longer outputs never help other models or tasks. The analysis run archives every completion and source hash. The earliest two reports predate the optional `split_seed` and `train_num_count` fields; absent values were normalized to their unchanged `None` defaults. All other settings matched, canonical puzzle sets were identical, and scores were recomputed before aggregation.

7 Pilot generalization on frozen suites

The original mixed-arity pilot was evaluated with 256-token greedy decoding on four solver-verified, frozen suites, each containing 256 puzzles and no overlap with its selected training puzzles. The

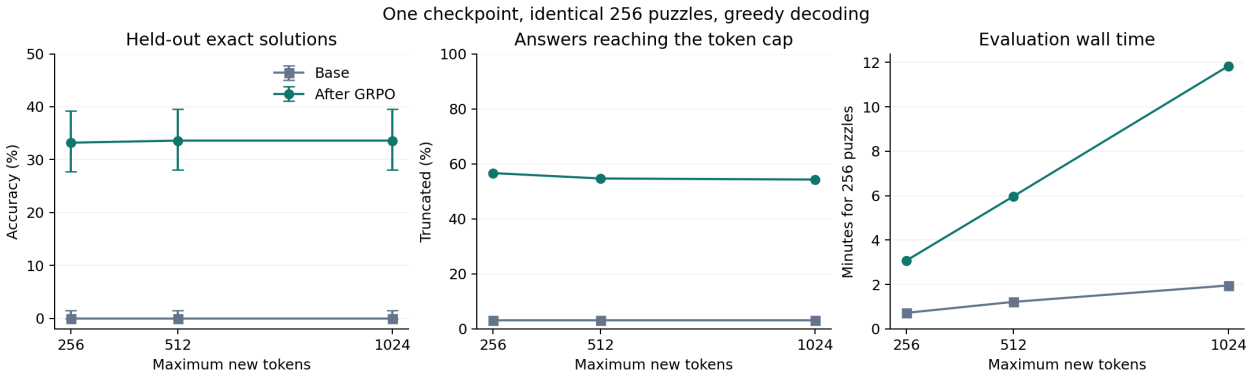


Figure 2: Complete matched budget grid. Error bars are 95% Wilson intervals over puzzles for one checkpoint, not training-seed variation.

base model scored zero strict solutions on every suite. The trained checkpoint's results were:

Suite	Solved	Accuracy (95% Wilson)	Truncated
Three-number holdout	146/256	57.0% (50.9–62.9)	91/256
Four-number holdout	41/256	16.0% (12.0–21.0)	184/256
Inputs 101–200	61/256	23.8% (19.0–29.4)	165/256
Targets 101–200	123/256	48.0% (42.0–54.2)	122/256

Diagnostic scores equal strict scores for the trained checkpoint. For the base, diagnostic extraction recovers one solution in each shifted suite, versus zero strict solutions; it recovers none in the two arity holdouts.

The 41-percentage-point three/four-number gap indicates strong sensitivity to task complexity under this evaluation. Number-multiset errors account for 176/256 four-number outputs. This describes observed errors, not a causal explanation of the gap. The arity holdouts may overlap the original pilot evaluation, so they are not independent replications and should not be pooled with that test set. The shifted suites balance three/four-number inputs and sample targets uniformly from reachable integers within their designated ranges. They are constructed distributions, not difficulty-matched versions of the original dataset. In particular, the 48% larger-target score does not establish superior out-of-distribution reasoning. Full per-answer evidence, hash validation, and paired model comparisons are in the generalization analysis. Training only on three-number puzzles and testing transfer is a separate, completed experiment reported below. Its aggregator requires all three training seeds and all 15 evaluations, checks that every training puzzle contains exactly three numbers, verifies all 300 steps and identical ordered splits, and re-scores every answer against the frozen suites. An end-to-end synthetic test rejects four-number training contamination before publishing output. Per-seed comparisons with the single mixed-arity pilot are descriptive; they are not a balanced multi-seed ablation of the training distribution.

8 Three-number-only training and transfer

All three seeds completed 300 training steps and five evaluations each. Training took 98.7, 91.1, and 100.3 minutes for seeds 42, 43, and 44. The table reports solved counts per checkpoint and the mean accuracy with sample standard deviation across seeds. This SD is not a confidence interval; the same puzzles recur across seeds and are not pooled as independent observations.

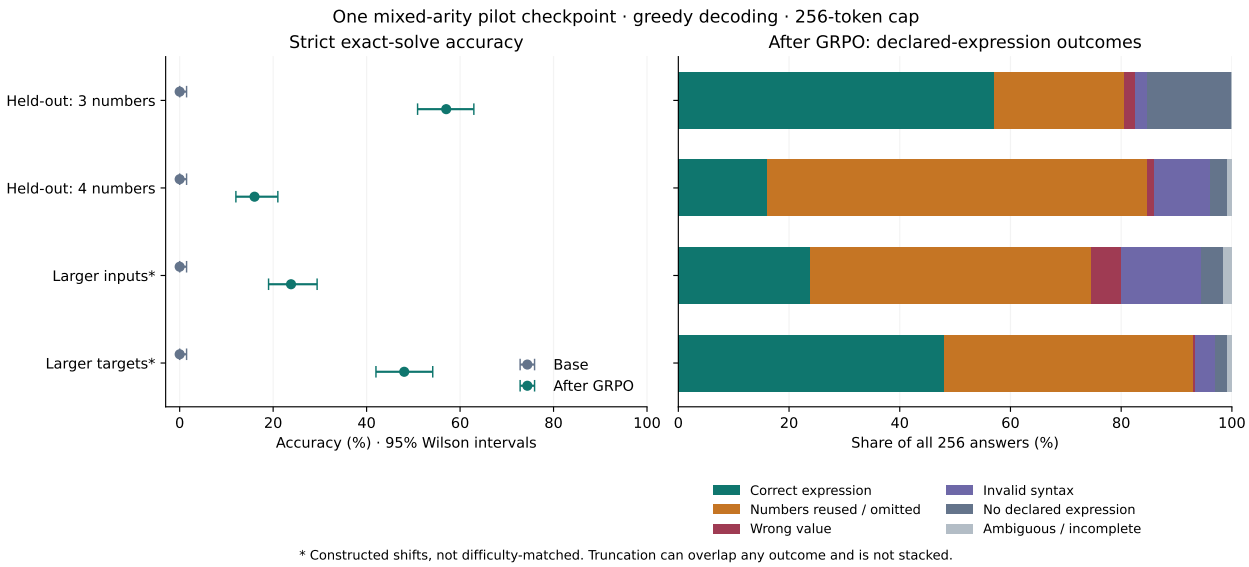


Figure 3: Strict solve accuracy and diagnostic outcomes for the single mixed-arity pilot. Error bars describe per-checkpoint puzzle uncertainty, not seed variation. Truncation overlaps diagnostic categories and is reported separately in the table.

Suite	Seed 42	Seed 43	Seed 44	Mean ± SD
Original holdout	79/256	93/256	77/256	32.4% ± 3.4 pp
Three-number	146/256	161/256	133/256	57.3% ± 5.5 pp
Four-number	26/256	38/256	23/256	11.3% ± 3.1 pp
Inputs 101–200	63/256	79/256	69/256	27.5% ± 3.2 pp
Targets 101–200	107/256	112/256	108/256	42.6% ± 1.0 pp

Strict and diagnostic scores agree in all 15 cells. Full step sequences, configuration, 20,000 unique three-number training puzzles, identical ordered splits, frozen suite hashes, absence of training overlap, and all per-answer scores passed audit. Final adapters and resumable checkpoints are retained in W&B. Raw completions, per-seed summaries, failure categories, provenance, and evaluation links are available in the complete evidence directory and analysis run.

Every checkpoint has lower accuracy on four-number puzzles. This is an observed transfer limitation under a 256-token cap, not an isolated causal effect of training arity. Comparisons with the single mixed-arity pilot remain descriptive; they are not a balanced multi-seed training-distribution ablation. The larger-input and larger-target suites are constructed and not difficulty-matched.

Three-number outputs hit the cap on 110, 95, and 119 of 256 puzzles for seeds 42, 43, and 44; four-number outputs do so on 211, 197, and 219 puzzles. Truncation overlaps failure categories and does not establish causation or show that a longer budget would solve the puzzles. The completed longer-output experiment used the mixed-arity pilot; its finding cannot establish the budget effect for these separately trained adapters.

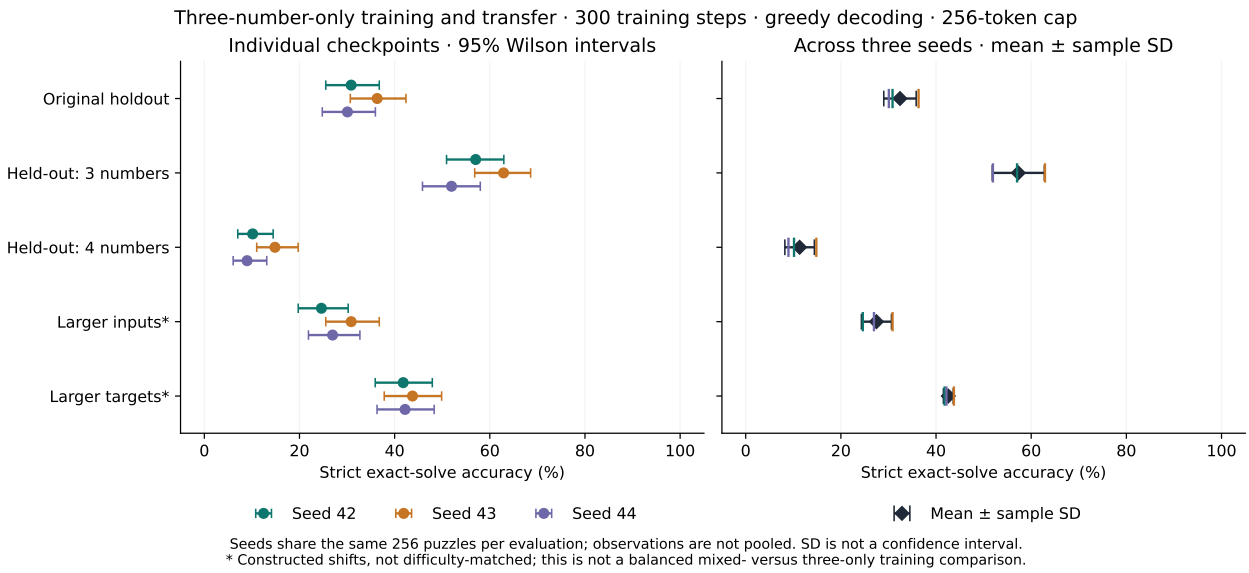


Figure 4: Completed transfer study. Individual checkpoint Wilson intervals and across-seed dispersion are shown separately. Shifted suites are constructed, not difficulty-matched.

9 Kernel validation and controlled-study backend

An A100 hardware trial completed three steps at 49.33 seconds per step, compared with the L4's then-current 50.79 seconds per step. This approximately 3% gain did not justify switching the controlled grid. The A100 run was intentionally interrupted, its complete partial evidence archived, and its VM released. It is excluded from completed training comparisons. These few steps are a throughput trial, not a reliable general hardware benchmark.

The kernel investigation archives the probes, successful five-step trial, and interrupted fallback evidence. After the hardware trial, FLA 0.5.2 was installed in an isolated directory and compared with unwrapped Transformers reference kernels at representative model shapes. A probe-only positional-argument error initially passed `beta` as `gk` to the recurrent function; inspection of the actual signatures and Transformers calls identified and corrected it. This was not a training failure. The corrected probe found about 3.27× chunk-forward/backward and 2.11× recurrent-forward microkernel speedups. Chunk output relative L2 difference was 0.44%, and gradient differences were 0.27–0.53%; the single recurrent output matched exactly. Returned states were checked for finiteness, not long-horizon equivalence. Compilation is excluded and concurrent GPU work limits the precision of these microtimings.

A separate five-step full-training trial completed in 203.3 seconds, with steps 2–5 averaging 37.63 seconds versus the fallback's approximately 50.8 seconds. All five sparse rewards were zero, so the test establishes operational viability, not learning quality. The original training process resumed automatically after a recorded 266.62-second suspension. On this evidence, the fallback attempt was subsequently stopped after 41 steps and archived, and all six controlled reward runs were restarted consistently with the pinned FLA backend in `controlled-l4-fla-v1`. Neither short trial enters the completed reward comparison. Original pilot and local transfer runs retain their fallback backend; this distinction must be retained in interpretation and reproduction.

10 Interactive comparison demo

A local browser interface exposes all 256 archived evaluation pairs, including failures, with filters for outcomes and number-multiset errors. An exact answer checker accepts user expressions and shows strict success, diagnostic success, the extracted expression, exact value, and missing or reused inputs. Archived mode uses only Python's standard library and requires no model download or GPU. An optional live mode loads the pilot LoRA adapter and compares greedy generation with the adapter enabled and disabled. Real CUDA generation passed HTTP and browser verification after the training queue finished. The server binds to loopback and serializes live inference requests.

Chromium checks verified all 256 examples, 85-success/171-failure filtering, interactive answer checking, disabled generation in archived mode, no JavaScript errors, and mobile layout without horizontal overflow at 390px width. HTTP tests also check invalid inputs and cross-origin request rejection. Browser checks are reproducible with `scripts/check_demo_browser.py`; these do not substitute for testing live model inference. A separate real-model check did verify repeated greedy base/adapter outputs, the browser generation button, rendered verifier details, mobile layout, and absence of JavaScript errors. The live verification run archives screenshots and exact outputs. On its single test puzzle, the base expression evaluates to 1274/19 rather than the requested 66; the adapter returns a valid expression equal to 66. This is an integration check, not a benchmark or additional accuracy evidence. Single-puzzle generation can differ from the historical batched evaluation, and custom puzzles are not guaranteed solvable.

11 Provenance and experiment ledger

The pilot used clean source commit `0a432a272591f7683b711a9c867c01e7d939fca1`. W&B contains source snapshots, configs, package versions, puzzle splits, checkpoints, and generation tables:

- Pilot baseline
- Pilot training
- Pilot final evaluation
- Final pilot adapter artifact
- Study workspace
- Archived-output diagnostic audit

The repository ledger records implementation and execution milestones. Current CPU coverage is 48 passing tests, including diagnostic extraction, exact reward validation, statistical analysis, demo HTTP behavior, schema compatibility, and remote archive-integrity checks. These tests do not substitute for the unfinished training grid or live model/browser validation.

An automatic collector preserves each completed remote job and exports W&B hardware and software provenance. The dedicated L4 VM is released only after all six completed jobs are downloaded and validated; aggregate analysis then runs locally. These safeguards do not imply that pending experiments have completed.

Local transfer source audit. The first transfer run reports a dirty working tree at commit `a2a8e9e39abab513192b79a7f161aab271823efe`. All 108 files in its W&B source snapshot match that recorded commit byte-for-byte; the artifact excludes untracked files, so the dirty flag is retained. Training and evaluation modules also match comparison commit `34faa55`. The only changed package module is the logging module, which adds Python and optional-kernel metadata. The first run predates those metadata fields; startup logs explicitly identify the fallback kernels. Hashes and audit scope are committed in `results/study/local-source-audit.json`.

Seed 43's 149 archived files match its clean recorded commit, and its hardware and normalized requirements match seed 42. Seed 44's 180 archived files match its clean startup commit; all package Python modules, hardware, Python version, and full normalized requirements match seed 43. The startup audit was followed by verification of finished training, the final adapter and resumable checkpoint, and all five evaluations. Finished runtime provenance is included with the complete transfer evidence. Per-seed source hashes and captured runtime evidence are retained in the repository's `local-source-audit-seed43.json` and `local-source-audit-seed44.json` files under `results/study`.

12 Limitations

The original pilot is one seed on a 256-puzzle holdout. The three-seed transfer grid is complete, but the six-run controlled reward grid remains incomplete. These results do not establish robust generalization. Training-dataset deduplication cannot rule out exposure in pretraining. Strict success conflates arithmetic correctness with protocol compliance; the diagnostic is an imperfect post-hoc measure, not a causal decomposition. Long answers may contain useful work but still fail due to truncation. The completed output-budget finding applies to this checkpoint and test set. Three transfer seeds quantify limited seed variation under one training budget and fixed evaluation suites; they do not establish robustness across arbitrary training distributions or tasks.